

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Engenharia de Software - LDAMD

Relatório de Integração

MOM (RabbitMQ)

Sprint 2 - Middleware Orientado a Mensagens | 25/05/2026

Disciplina	Lab. de Desenvolvimento de Aplicações Móveis e Distribuídas
Professores	Cleiton Silva Tavares e Cristiano de Macedo Neto
Entrega	Sprint 2 - 25/05/2026

1. Escolha da Ferramenta: RabbitMQ

O RabbitMQ foi escolhido como Middleware Orientado a Mensagens (MOM) por ser a ferramenta citada como exemplo no próprio enunciado do projeto e por ser amplamente adotada na indústria para comunicação assíncrona entre serviços.

Suas principais características que motivaram a escolha são:

- **Protocolo AMQP 0-9-1:** protocolo aberto e robusto, com garantia de entrega e suporte a filas duráveis.
- **Painel de gerenciamento (Management UI):** interface web na porta 15672 que permite visualizar filas, mensagens e consumidores em tempo real, essencial para demonstrar o funcionamento durante a apresentação.
- **Mensagens persistentes:** com `delivery_mode = 2` as mensagens sobrevivem a reinicializações do broker.
- **Instalação local simples:** disponível para Windows, macOS e Linux com poucos comandos.
- **Biblioteca pika:** cliente Python oficial, estável e bem documentado.

2. Padrão Utilizado: Default Exchange (Direct Routing)

O padrão adotado utiliza o Default Exchange do RabbitMQ, que implementa roteamento direto (direct routing) baseado na `routing_key`.

Neste padrão, o producer publica mensagens com uma `routing_key` igual ao nome da fila de destino, e o broker entrega diretamente àquela fila. Os consumers se inscrevem

em filas específicas e processam as mensagens de forma assíncrona, sem qualquer chamada REST direta entre producer e consumer.

Este padrão foi escolhido por sua simplicidade e aderência ao domínio: cada tipo de evento tem sua própria fila nomeada (`order.created` , `order.status_updated` , `product.sold_out`), o que torna o fluxo de eventos explícito, rastreável e fácil de demonstrar.

Fluxo de publicação:

- 1. Comprador faz POST `/api/v1/orders`
- 2. Service cria o pedido no banco
- 3. Service chama `publish_order_created(order)`
- 4. Publisher abre conexão AMQP e publica na fila `order.created`
- 5. Consumer recebe e processa de forma assíncrona

3. Documentação dos Eventos

Evento	Producer	Consumer	Disparado quando	Fila
<code>order.created</code>	OrderService (Flask)	RabbitMQ Consumer	Comprador realiza um novo pedido com sucesso	<code>order.created</code>
<code>order.status_updated</code>	OrderService (Flask)	RabbitMQ Consumer	Vendedor atualiza o status do pedido	<code>order.status_updated</code>
<code>product.sold_out</code>	OrderService (Flask)	RabbitMQ Consumer	Estoque do produto chega a zero após um pedido	<code>product.sold_out</code>

4. Payloads JSON de Exemplo

Evento: `order.created`

```
{
  "event": "order.created",
  "data": {
    "order_id": 1,
    "buyer_id": 2,
    "product_id": 1,
    "quantity": 2,
    "status": "pending"
  }
}
```

Evento: order.status_updated

```
{
  "event": "order.status_updated",
  "data": {
    "order_id": 1,
    "buyer_id": 2,
    "product_id": 1,
    "previous_status": "pending",
    "new_status": "confirmed"
  }
}
```

Evento: product.sold_out

```
{
  "event": "product.sold_out",
  "data": {
    "product_id": 1,
    "seller_id": 1
  }
}
```

```
* Debugger PIN: 119-582-122
127.0.0.1 - - [25/May/2026 19:45:17] "PATCH /api/v1/orders/3/status HTTP/1.1" 400 -
Evento enviado: order.created
127.0.0.1 - - [25/May/2026 19:45:28] "POST /api/v1/orders HTTP/1.1" 201 -
□
```

```
Evento recebido:
{
  "event": "order.created",
  "data": {
    "order_id": 4,
    "buyer_id": 2,
    "product_id": 1,
    "quantity": 1,
    "status": "pending"
  }
}
```

5. Decisões de Design

- **Falha silenciosa no publisher:** O publisher captura exceções de conexão com o RabbitMQ e registra em log, mas não propaga o erro para o controller. Isso garante que uma falha momentânea no broker não derrube o endpoint REST. A operação no banco já foi confirmada; o evento pode ser republicado em uma futura estratégia de retry.
- **Mensagens persistentes (delivery_mode=2):** Toda mensagem publicada usa `delivery_mode=2`, que instrui o RabbitMQ a gravar a mensagem em disco antes de confirmar. Combinado com filas `durable=True`, isso garante que nenhuma mensagem seja perdida em caso de reinicialização do broker.
- **Filas nomeadas por evento:** Cada evento tem sua própria fila (`order.created`, `order.status_updated`, `product.sold_out`) em vez de uma fila genérica. Isso permite que consumers diferentes se inscrevam apenas nos eventos de seu interesse, seguindo o princípio de segregação de responsabilidades.
- **Consumer com reconexão automática:** O consumer implementa um loop de reconexão com espera de 5 segundos entre tentativas. Isso o torna resiliente a reinicializações do broker durante desenvolvimento e demonstrações.
- **ACK explícito por mensagem:** O consumer usa `basic_ack` após processar cada mensagem com sucesso e `basic_nack` em caso de erro. O RabbitMQ só remove a mensagem da fila após o ACK, garantindo que mensagens com erro sejam reprocessadas (`requeue=True`) ou descartadas (`requeue=False` para JSON inválido).

6. Desafios Encontrados

- **Importação circular:** integrar o publisher ao service sem criar dependências circulares entre módulos. **Solução:** criar a pasta `messaging/` com `publisher.py` independente, importado pelo service após a inicialização do app.
- **RabbitMQ não disponível:** garantir que o backend não quebre se o RabbitMQ não estiver rodando. **Solução:** captura de `AMQPConnectionError` no publisher com log

de erro sem relançamento da exceção.

- **Contexto Flask no consumer:** o consumer roda fora do app Flask, sem acesso ao banco. **Solução:** consumer atua apenas como processador de notificações (log + push), sem acesso direto ao banco de dados.